

Peregrine Framework User Guide

Welcome to the comprehensive user guide for the **Peregrine C++ Web Framework**. This guide is designed to take you from a basic "Hello World" application to building complex, modular web architectures in C++.

If you are coming from Python's Flask, the concepts and API design here will feel incredibly familiar.

1. Installation & Project Setup

Peregrine is designed to be lightweight and easy to integrate into existing C++ projects.

Prerequisites

- **C++ Compiler:** Must support C++17 or higher (GCC, Clang, MSVC).
- **OpenSSL:** Required for secure sessions and HTTPS support.
- **Pthreads:** Required on POSIX systems for the multi-threaded server.

Including in your project

Since Peregrine provides its headers in `include/peregrine/`, you simply need to include this directory in your build system.

Example `CMakeLists.txt` :

```
cmake_minimum_required(VERSION 3.10)
project(MyPeregrineApp)

set(CMAKE_CXX_STANDARD 17)

# Find OpenSSL and Threads
find_package(OpenSSL REQUIRED)
find_package(Threads REQUIRED)

include_directories(include)

add_executable(app main.cpp)
target_link_libraries(app OpenSSL::SSL OpenSSL::Crypto Threads::Threads)
```

2. Quickstart

To initialize Peregrine, you include the master header and create an `App` (or `Peregrine`) instance.

```
#include "peregrine/peregrine.hpp"

using namespace peregrine;

int main() {
    Peregrine app("my_app");

    // A simple GET route
    app.get("/", [](Request& req) {
        return Response::html("<h1>Welcome to Peregrine!</h1>");
    });

    // Start the server on port 8080
    app.run("0.0.0.0", 8080);
    return 0;
}
```

[!NOTE] Unlike Python, C++ is compiled. You will need to recompile and restart your server to see changes unless you implement a separate hot-reloading mechanism.

3. Routing Deep Dive

Peregrine maps HTTP requests to your C++ lambda functions (or standard functions) using routing methods.

HTTP Methods

You can map specific HTTP verbs directly:

```
app.get("/login", [](Request& req) { ... });
app.post("/login", [](Request& req) { ... });
app.put("/update", [](Request& req) { ... });
app.del("/delete", [](Request& req) { ... });
```

Or use the generic `route` method to accept multiple methods:

```
app.route("/api/data", {"GET", "POST"}, [](Request& req) {
    if (req.method == "POST") {
        // Handle post
    }
    return Response::json({{"status", "ok"}});
});
```

```
});
```

Path Parameters (Variables)

You can extract variables directly from the URL. Peregrine stores these in the `req.path_params` map.

```
app.get("/user/<int:user_id>", [](Request& req) {
    std::string id = req.path_params["user_id"];
    return Response::text("Showing profile for user: " + id);
});
```

Supported types (used for regex matching internally):

- `<string:var>` or `<var>` : Matches any text up to a slash.
- `<int:var>` : Matches digits only.
- `<path:var>` : Matches any text, including slashes.

4. The Request & Response Objects

Because C++ does not have a global runtime interpreter, there is no global `request` object. Instead, every route handler is explicitly passed a `Request& req` reference.

Parsing the Request

```
app.post("/submit", [](Request& req) {
    // Query string: ?sort=desc
    std::string sort = req.arg("sort", "asc"); // "asc" is default

    // Form data (application/x-www-form-urlencoded)
    std::string username = req.form_get("username");

    // JSON Payload
    if (req.is_json()) {
        Json data = req.get_json();
        std::string email = data["email"].string_value();
    }

    // Headers
    std::string auth = req.header("Authorization");

    return Response::text("Received");
});
```

File Uploads

Peregrine handles `multipart/form-data` natively.

```
app.post("/upload", [(Request& req) {
    for (const auto& file : req.files) {
        if (file.field_name == "profile_pic") {
            file.save("/uploads/" + file.filename);
        }
    }
    return Response::text("Uploaded successfully");
}]);
```

Creating Responses

Use the static factory methods on the `Response` class for ergonomic replies.

```
// JSON Response (Content-Type: application/json)
return Response::json({
    {"id", 123},
    {"status", "active"}
});

// Redirect
return Response::redirect("/dashboard", 302);

// Customizing Headers and Status
Response res = Response::text("Created");
res.set_status(201);
res.set_header("X-Custom-Header", "Peregrine");
return res;
```

5. Secure Sessions & Cookies

Peregrine implements a secure, cryptographically signed cookie session engine identical to Flask's.

Setting up Sessions

You **must** provide a `secret_key` to use sessions.

```
app.secret_key = "super_secret_unguessable_string";
// Optional: allow key rotation without logging users out
app.secret_key_fallbacks = {"old_secret_key"};
```

Using Sessions

The session data is stored in `req.session.data()`, which is a map of strings.

```
app.post("/login", [](Request& req) {
    req.session.data()["user_id"] = "456";
    req.session.permanent = true; // Makes cookie last `session_lifetime_seconds`
    return Response::redirect("/profile");
});

app.get("/profile", [](Request& req) {
    if (req.session.data().count("user_id") == 0) {
        return Response::redirect("/login");
    }
    std::string uid = req.session.data()["user_id"];
    return Response::text("Hello User " + uid);
});
```

[!WARNING] Because sessions are stored in cookies on the client side, **do not store sensitive information like passwords in the session**. The contents are signed (tamper-proof) but not encrypted (they can be read by the client).

6. Blueprints for Modular Apps

As your application grows, writing all routes in `main.cpp` becomes impossible to maintain. Use **Blueprints** to split your app into modules.

In `api.hpp`:

```
#include "peregrine/blueprint.hpp"

inline peregrine::Blueprint create_api_blueprint() {
    peregrine::Blueprint api("api", "/api/v1");

    api.get("/status", [](peregrine::Request& req) {
        return peregrine::Response::json({{"status", "online"}});
    });

    return api;
}
```

In `main.cpp`:

```
#include "api.hpp"

int main() {
```

```
Peregrine app("my_app");

Blueprint api_bp = create_api_blueprint();
app.register_blueprint(api_bp);

app.run("0.0.0.0", 8080);
return 0;
}
```

7. Middleware & Error Handling

Before & After Requests

You can register global hooks that run on every request. This is useful for authentication, database connection management, or logging.

```
app.before_request([](Request& req) {
    // Store data in the per-request `g` context
    req.g["start_time"] = get_current_time();

    // Note: In Peregrine, if you want to abort the request early,
    // you throw a HTTPException rather than returning a response.
    // (Throwing exceptions allows the hook to immediately halt execution).
});

app.after_request([](Request& req, Response& res) {
    res.set_header("X-Powered-By", "Peregrine C++");
});
```

Custom Error Pages

Catch HTTP errors globally:

```
app.error_handler(404, [](Request& req) {
    return Response::html("<h1>Oops, we couldn't find that!</h1>", 404);
});

app.error_handler(500, [](Request& req) {
    return Response::json({"error", "Internal Server Error"}, 500);
});
```

What's Next?

Peregrine gives you the building blocks for modern web architecture. Combined with a powerful C++ ORM or database driver, you can build incredibly fast, type-safe web applications that are just as easy to maintain as their Python equivalents.